

UNIT – 2

PROCESS MANAGEMENT

- 2.1 Define process and process control block
- 2.2 Explain process state diagram
- 2.3 Describe process creation and termination
- 2.4 Discuss the relation between processes
- 2.5 Define Thread and describe multithreading
- 2.6 Explain scheduling concepts
- 2.7 Describe scheduling queues and schedulers
- 2.8 Explain CPU scheduling and scheduling criteria
- 2.9 Explain various scheduling algorithms
 - 2.9.1 FCFS
 - 2.9.2 SJF
 - 2.9.3 Round Robin
 - 2.9.4 Priority
 - 2.9.5 Multilevel Scheduling

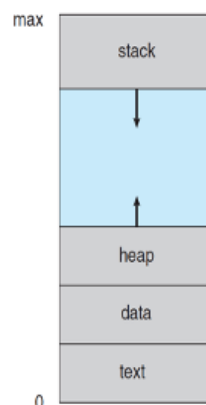
2.0 What is Process Management

- One of the important task of the operating system is to manage the resources.
- Most important resource of the system is CPU (Processor).
- So operating system must use the CPU in an effective way.
- In single user system, the processor is busy only when the user is executing a job or program, at all other times it is idle.
- However when they are many users with many jobs on the system, the processor must be allocated to each job in an effective manner which can be a complex task.
- The CPU can execute only one instruction at a time and that instruction can belong to only one of the programs residing in the memory.
- The operating system will have to allocate the CPU time to various users based on certain policy. This is done by process management.

Basic Concepts Of Process

A process is essentially running software. The execution of any process must occur in a Specific order. A process is defined as an entity which represents the basic unit of work to be implemented in the system.

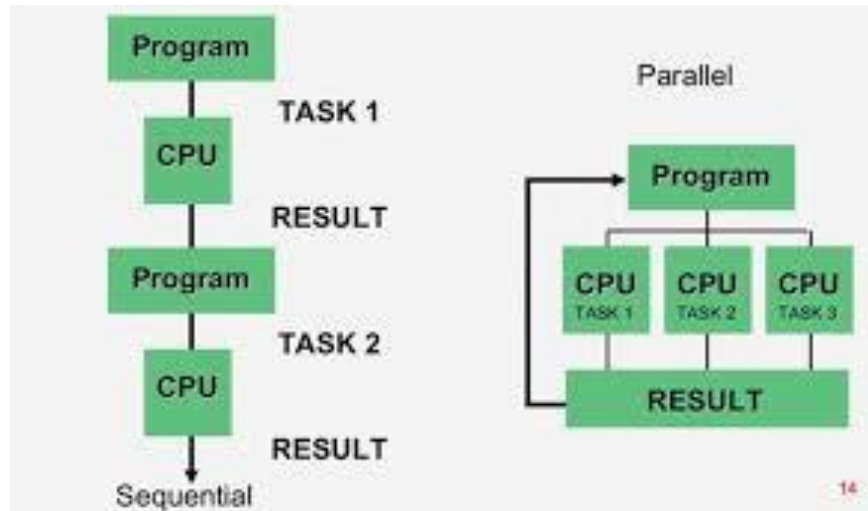
When a program is loaded into the memory and it becomes a process, it can be divided into of a process inside main memory:



1. Stack: The process Stack contains the temporary data such as method/function parameters, return address and local variables.
2. Heap: This is dynamically allocated memory to a process during its run time.
3. Text: This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.
4. Data: This section contains the global and static variables.

Sequential Processes:

A sequential process is a process that executes one instruction at a time in a specific order. It does not interact with other processes during execution and runs independently. This type of process is simple and easy to manage but does not support multitasking.

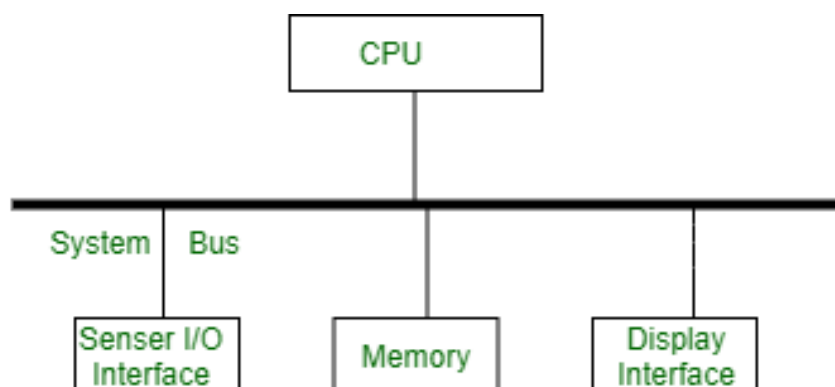


Parallel Processing :

Parallel processing refers to the simultaneous execution of multiple processes or threads using multiple CPUs or cores. It increases system speed and performance by handling tasks at the same time. It is commonly used in high-performance computing and multi-core systems.

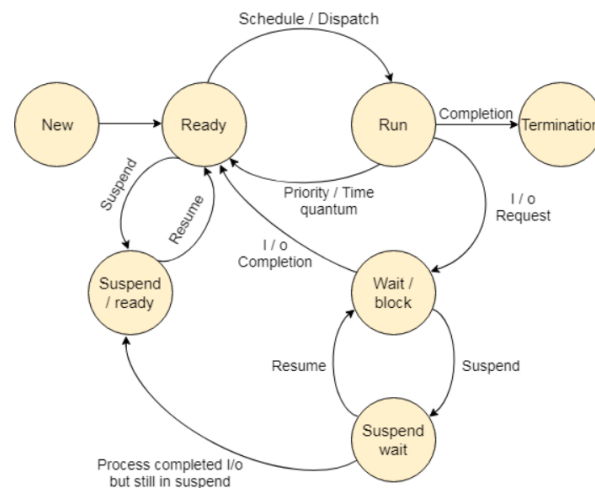
Concurrent Processes:

Concurrent processes are multiple processes that progress during the same time period. They may not run exactly at the same moment but are managed by the operating system using techniques like time-sharing. These processes may need synchronization and communication to work correctly.



Process State Diagram

The process, from its creation to completion, passes through various states. The minimum number of states is five. The names of the states are not standardized although the process may be in one of the following states during execution.



1. New

- A program which is going to be picked up by the OS into the main memory is called a new process.

2. Ready

- Whenever a process is created, it directly enters in the ready state, in which, it waits for the CPU to be assigned.
- The OS picks the new processes from the secondary memory and put all of them in the main memory.
- The processes which are ready for the execution and reside in the main memory are called ready state processes.
- There can be many processes present in the ready state.

3. Running

- One of the processes from the ready state will be chosen by the OS depending upon the scheduling algorithm.
- Hence, if we have only one CPU in our system, the number of running processes for a particular time will always be one.
- If we have n processors in the system then we can have n processes running simultaneously.

4. Block or wait

- From the Running state, a process can make the transition to the block or wait state depending upon the scheduling algorithm or the intrinsic behavior of the process.
- When a process waits for a certain resource to be assigned or for the input from the user then the OS move this process to the block or wait state and assigns the CPU to the other processes.

5. Completion or termination

- When a process finishes its execution, it comes in the termination state. All the context of the process (Process Control Block) will also be deleted the process will be terminated by the Operating system.

6. Suspend ready

- A process in the ready state, which is moved to secondary memory from the main memory due to lack of the resources is called in the suspend ready state.
- If the main memory is full and a higher priority process comes for the execution then the OS have to make the room for the process in the main memory by throwing the lower priority process out into the secondary memory.
- The suspend ready processes remain in the secondary memory until the main memory gets available.

7. Suspend wait

- Instead of removing the process from the ready queue, it's better to remove the blocked process which is waiting for some resources in the main memory.
- Since it is already waiting for some resource to get available hence it is better if it waits in the secondary memory and make room for the higher priority process.
- These processes complete their execution once the main memory gets available and their wait is finished

Process Control Block

A Process Control Block (PCB) is a data structure used by the operating system to store information about a specific process. It acts like the identity card of a process and contains all the details needed to manage and control that process. Key information stored in a PCB includes:



1. Process ID (PID)

Every process is assigned a unique Process ID by the operating system. This helps the OS to identify and manage each process separately.

2. Process State

The PCB keeps track of the current state of the process, such as new, ready, running, waiting, or terminated. This helps the OS know what the process is doing.

3. Program Counter

This stores the address of the next instruction the process will execute. It ensures the process resumes from the correct place after interruption.

4. CPU Registers

The contents of all CPU registers used by the process are saved in the PCB. These are restored during context switching to resume execution properly.

5. Memory Management Information

It contains details like base and limit registers or page tables that define the process's memory space. This helps in memory protection and allocation.

6. Scheduling Information

This includes priority level, scheduling queue pointers, and other data used by the scheduler to decide the order of process execution.

7. I/O Status Information

The PCB tracks the list of I/O devices used and files opened by the process. This helps the OS manage input/output operations for the process.

Process Creation And Termination

Process Creation and Process termination are used to create and terminate processes respectively. Details about these are given as follows.

Process Creation

Process creation means starting a new process by the operating system. It happens when a program is loaded into memory and begins execution.

In an operating system, a new process can be created in the following ways:

- 1. User Request for Process Creation**

When a user starts a program or application (like clicking an icon), the operating system creates a new process to run that program.

- 2. System Initialization**

During system startup, the OS automatically creates several background processes needed for system operations.

- 3. Batch Job Initialization**

In batch processing systems, processes are created to execute jobs that are submitted in batches, usually without user interaction.

- 4. Execution of a Process Creation System Call by a Running Process**

A running process can create a new process using system calls like `fork()` in UNIX.

This is common in multitasking systems where a process may need to run a sub-process.

Process Termination

Process termination occurs when the process is terminated. The `exit()` system call is used by most operating systems for process termination.

Some of the causes of process termination are as follows –

- A process may be terminated after its execution is naturally completed. This process leaves the processor and releases all its resources.
- A child process may be terminated if its parent process requests for its termination.
- A process can be terminated if it tries to use a resource that it is not allowed to. For example - A process can be terminated for trying to write into a read only file.
- If an I/O failure occurs for a process, it can be terminated. For example - If a process requires the printer and it is not working, then the process will be terminated.
- If a process requires more memory than is currently available in the system, then it is terminated because of memory scarcity.

Relation Between Processes

Relation between processes means how processes are connected in an operating system. A parent process can create child processes, and those children are called siblings. Processes may be independent or cooperate by sharing data and resources.

In an operating system, processes can be related to each other in different ways:

1. Parent-Child Relationship

When a process creates another process, the new one is called the **child**, and the creator is the **parent**. The child may inherit some properties from the parent.

2. Cooperating Processes

These processes work together to perform a task. They can share data and resources and communicate using Inter-Process Communication (IPC).

3. Independent Processes

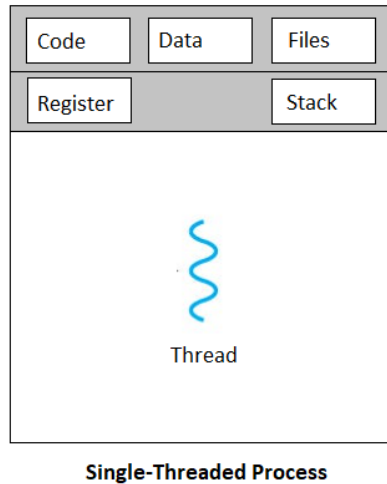
These processes run separately and do not share data or resources. They do not affect each other's execution.

4. Sibling Processes

If two or more processes are created by the same parent, they are called sibling processes.

Threads

A thread is the smallest unit of execution within a process. A single process can have multiple threads, and they all share the same memory space but can run independently. Threads are used to perform multiple tasks at the same time (concurrent execution) within the same program, which improves speed and efficiency.



For example, in a web browser:

- One thread loads the page
- Another plays a video
- Another handles user clicks

Advantages of Threads:

1. Faster context switching compared to processes.
2. Efficient resource sharing among threads of the same process.
3. Useful for multitasking within an application.
4. Less overhead than creating new processes.
5. Improves program responsiveness

Disadvantages of Threads:

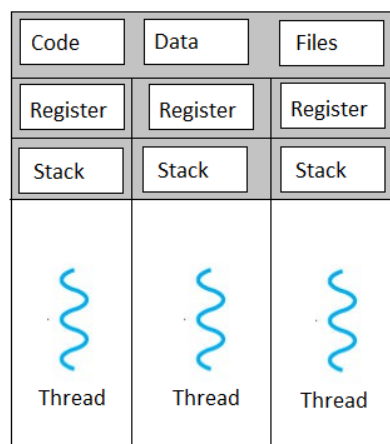
1. If one thread crashes, it may affect the whole process.
2. Debugging multi-threaded programs is complex.
3. Requires careful synchronization to avoid data conflicts.
4. Can lead to deadlocks if not managed properly.
5. Not all systems fully support multi-threading.

Processor Vs Threads

S.N	Process	Thread
1.	Process is heavy weight or resource intensive	Thread is light weight, taking lesser resources than a process
2.	Process switching needs interaction with operating system	Thread switching does not need to interact with operating system.
3.	In multiple processing environments, each process executes the same code but has its own memory and file resources.	All threads can share same set of open files. child processes.
4.	If one process is blocked, then no other process can execute until the first process is unblocked.	While one thread is blocked and waiting, a second thread in the same task can run.
5.	Multiple processes without using threads use more resources.	Multiple threaded processes use fewer resources.
6.	In multiple processes each process operates independently of the others	One thread can read, write or change another thread's data

Multi threading

It is a process of multiple threads executes at same time. Many operating systems support kernel thread and user thread in a combined way. Example of such system is Solaris. Multi threading model are of three types.



Multithreaded Process

Advantages of Multithreading:

1. Increases the performance of applications.
2. Better CPU utilization.
3. Faster program execution through parallelism.

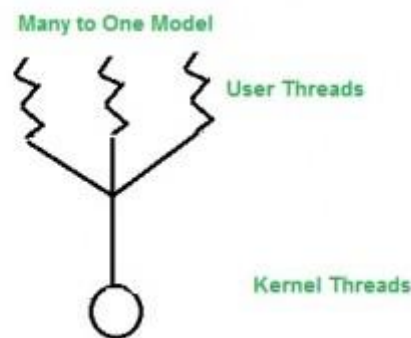
Disadvantages of Multithreading:

1. Difficult to program and manage.
2. Requires careful synchronization to avoid conflicts.
3. Bugs are hard to detect and fix.

Many-to-One Multithreading Model

In this model, **many user-level threads are mapped to a single kernel thread**. All thread management is done by the user-level thread library, and the OS kernel knows only about the one kernel thread. The main disadvantage is that if one thread makes a blocking system call, the whole process is blocked.

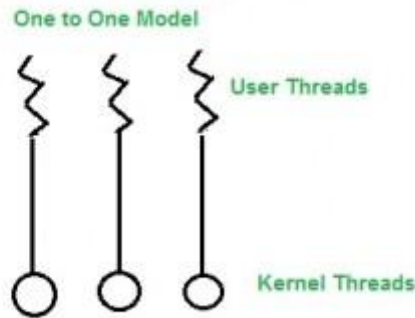
Example: Early Java threads (Green Threads) or some older UNIX systems.



One-to-One Multithreading Model

In this model, **each user thread is mapped to one kernel thread**. This allows multiple threads to run in parallel on multiple processors. It offers better concurrency but can be limited by system resources, as creating many kernel threads uses more memory.

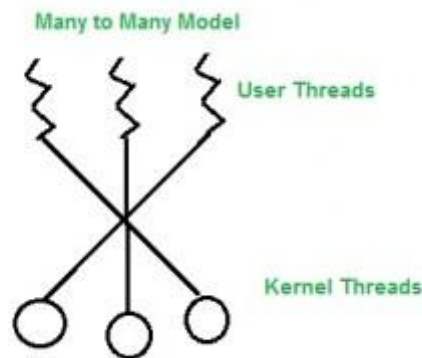
Example: Windows, Linux, and modern versions of Java use this model.



Many-to-Many Multithreading Model

This model allows **many user-level threads to be mapped to a smaller or equal number of kernel threads**. It gives flexibility in managing threads and avoids the blocking issue of the many-to-one model. Threads can run in parallel and are more efficiently managed.

Example: Solaris and Windows with thread pooling systems.



Scheduling Concepts

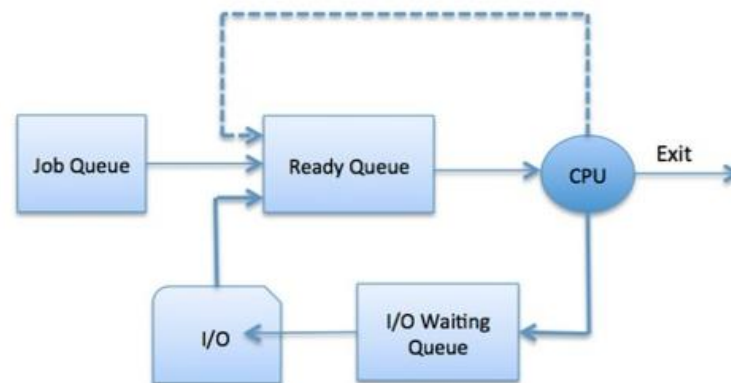
- ✓ Process scheduling is an essential part of a Multiprogramming operating systems.
- ✓ The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy.
- ✓ Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

Scheduling Queues And Schedulers

Process Scheduling Queues

The OS maintains all PCBs in Process Scheduling Queues. The OS maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are

placed in the same queue. When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.



The Operating System maintains the following important process scheduling queues

1. Job Queue:

- Contains all processes in the system.
- When a process is created, it's first added here

2. Ready Queue:

- Contains processes that are in main memory and ready to execute.
- These processes are waiting to be assigned to the CPU.
- Only one process is picked at a time for execution.

3. CPU:

- Executes processes from the ready queue.
- After execution, a process either:
- Exits (if it completes),
- Or moves to the I/O Waiting Queue if it needs I/O operations.

4. I/O Waiting Queue:

- Holds processes that are waiting for I/O devices (like disk, printer).
- These are in a blocked state.

5. I/O:

- When the I/O operation is done, the process is sent back to the Ready Queue for more CPU time.

Schedulers

Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run.

Schedulers are of three types

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler Long

Long-Term Scheduler

The long-term scheduler controls the admission of processes into the system from the job pool. It determines which processes are loaded into memory for execution, thereby controlling the degree of multiprogramming.

For example, in a batch processing system at a bank, the long-term scheduler may decide which batch jobs (like payroll processing) are allowed into the system during off-peak hours.

Short-Term Scheduler

The short-term scheduler, or CPU scheduler, decides which of the ready processes will be executed next by the CPU. It operates very frequently and must be fast.

A real-time example is in a smartphone OS, where the CPU scheduler rapidly switches between apps like WhatsApp, a music player, and background updates to ensure responsiveness.

Medium-Term Scheduler

The medium-term scheduler temporarily removes (swaps out) processes from main memory to reduce the load and then brings them back later (swapping). This is useful for managing memory when there are too many processes. In a desktop computer running heavy applications like Photoshop and a game simultaneously, the OS might swap out the background process to free up memory for the active game.

CPU Scheduling And Scheduling Criteria

CPU Scheduling

CPU scheduling It is a process that allows one process to use the CPU while the execution of another process is on hold (in waiting state) due to unavailability of any resource like I/O etc, thereby making full use of CPU. The aim of CPU scheduling is to make the system efficient, fast, and fair.

- Different CPU scheduling algorithms have different properties and the choice of a particular algorithm depends on the various factors.
- Many criteria have been suggested for comparing CPU scheduling algorithms.

Scheduling Criteria

Different CPU scheduling algorithms have different properties and the choice of a particular algorithm depends on various factors. Many criteria have been suggested for comparing CPU scheduling algorithms.

The criteria include the following:

CPU utilization: The main objective of any CPU scheduling algorithm is to keep the CPU as busy as possible. Theoretically, CPU utilization can range from 0 to 100 but in a real-time system, it varies from 40 to 90 percent depending on the load upon the system.

Throughput: A measure of the work done by the CPU is the number of processes being executed and completed per unit of time. This is called throughput. The throughput may vary depending on the length or duration of the processes.

Turnaround time: For a particular process, an important criterion is how long it takes to execute that process. The time elapsed from the time of submission of a process to the time of completion is known as the turnaround time. Turn-around time is the sum of times spent waiting to get into memory, waiting in the ready queue, executing in CPU, and waiting for I/O.

Waiting time: A scheduling algorithm does not affect the time required to complete the process once it starts execution. It only affects the waiting time of a process i.e. time spent by a process waiting in the ready queue.

Response time: In an interactive system, turn-around time is not the best criterion. A process may produce some output fairly early and continue computing new results while previous results are being output to the user. Thus, another criterion is the time taken from submission of the process of the request until the first response is produced. This measure is called response time.

Scheduling Algorithms

Scheduling algorithms can be classified into

Preemptive Scheduling:

Preemptive scheduling is used when a process switches from running state to ready state or from the waiting state to ready state. The resources (mainly CPU cycles) are allocated to the process for a limited amount of time and then taken away, and the process is again placed back in the ready queue if that process still has CPU burst time remaining. That process stays in the ready queue till it gets its next chance to execute.

Non-Preemptive Scheduling:

Non-preemptive Scheduling is used when a process terminates, or a process switches from running to the waiting state. In this scheduling, once the resources (CPU cycles) are allocated to a process, the process holds the CPU till it gets terminated or reaches a waiting state. In the case of non preemptive scheduling does not interrupt a process running CPU in the middle of the execution. Instead, it waits till the process completes its CPU burst time, and then it can allocate the CPU to another process.

A process scheduler schedules different process to be assigned to the CPU based on particular scheduling algorithm, the different scheduling algorithms are listed below:

- 1) First-Come, First-Served (FCFS) Scheduling
- 2) Shortest-Job-Next (SJN) Scheduling
- 3) Priority Scheduling
- 4) Shortest Remaining Time
- 5) Round Robin(RR) Scheduling
- 6) Multiple-Level Queues Scheduling

These algorithms are either non-preemptive or preemptive. Non-preemptive algorithms are designed so that once a process enters the running state, it cannot be preempted until it completes its allotted time, whereas the preemptive scheduling is based on priority where a scheduler may preempt a low priority running process anytime when a high priority process enters into a ready state.

First Come First Serve (FCFS)

- Jobs are executed on first come, first serve basis.
- It is a non-preemptive, pre-emptive scheduling algorithm.
- Easy to understand and implement.
- Its implementation is based on FIFO queue.
- Poor in performance as average wait time is high.

Problem:

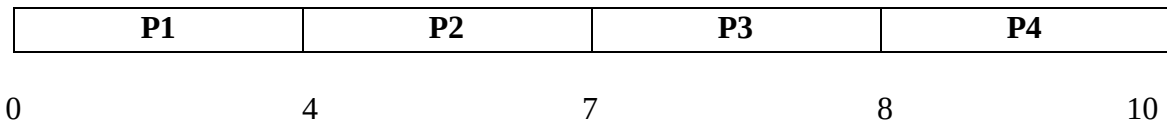
Consider the following set of processes with their arrival times and burst times:

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	4
P2	1	3
P3	2	1
P4	3	2

Using the **FCFS scheduling algorithm**, calculate:

1. Completion Time (CT) for each process
2. Turnaround Time (TAT) = CT - Arrival Time
3. Waiting Time (WT) = TAT - Burst Time
4. Average Turnaround Time and Average Waiting Time

1) Gantt chart



2) Solution:

We schedule the processes in the order they arrive:

- **P1** starts at time 0 → runs for 4 ms → completes at 4 ms
- **P2** starts at 4 ms → runs for 3 ms → completes at 7 ms
- **P3** starts at 7 ms → runs for 1 ms → completes at 8 ms
- **P4** starts at 8 ms → runs for 2 ms → completes at 10 ms

Now calculate the values:

Process	Arrival Time(AT)	Burst Time(BT)	Completion Time (CT)	TAT = CT - AT	WT = TAT - BT
P1	0	4	4	$4 - 0 = 4$	$4 - 4 = 0$
P2	1	3	7	$7 - 1 = 6$	$6 - 3 = 3$
P3	2	1	8	$8 - 2 = 6$	$6 - 1 = 5$
P4	3	2	10	$10 - 3 = 7$	$7 - 2 = 5$

Turnaround Time (TAT)

- **Average TAT** = $(4 + 6 + 6 + 7) / 4 = 5.75$ ms
- **Average WT** = $(0 + 3 + 5 + 5) / 4 = 3.25$ ms

Advantages of FCFS Scheduling:

1. Simple and easy to implement.
2. Fair in the sense that processes are handled in the order they arrive.
3. Suitable for batch systems where execution order is not time-sensitive.

Disadvantages of FCFS Scheduling:

1. Can cause long waiting times for shorter processes (convoy effect).
2. Non-preemptive nature makes it unsuitable for interactive or real-time systems.
3. Poor CPU utilization if a long process delays many short ones.

Shortest-Job-Next (SJN) Scheduling

- This is also known as shortest job first, or SJF
- This is a non-preemptive, pre-emptive scheduling algorithm.
- Best approach to minimize waiting time.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known
- The processor should know in advance how much time process will take.

Problem:

Consider the following set of processes with their arrival times and burst times:

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	8
P2	1	4
P3	2	2
P4	3	1
P5	4	3

Step-by-Step Execution:

Step 1: At Time = 0

Only **P1** has arrived → Execute **P1 (8 ms)**

Step 2: After P1 finishes at Time = 8

Processes arrived: **P2 (4 ms), P3 (2 ms), P4 (1 ms), P5 (3 ms)**

Shortest job = **P4 (1 ms)** → Execute **P4**

Step 3: After P4 finishes at Time = 9

Remaining: **P2 (4 ms), P3 (2 ms), P5 (3 ms)**

Shortest = **P3 (2 ms)** → Execute **P3**

Step 4: After P3 finishes at Time = 11

Remaining: **P2 (4 ms), P5 (3 ms)**

Shortest = **P5 (3 ms)** → Execute **P5**

Step 5: After P5 finishes at Time = 14

Remaining = **P2 (4 ms)** → Execute **P2**

Gantt Chart

P1	P4	P3	P5	P2	
0	8	9	11	14	18

Now calculate the values:

Process	Arrival Time(AT)	Burst Time(BT)	Completion Time(CT)	TAT = CT - AT	WT = TAT - BT
P1	0	8	8	$8 - 0 = 8$	$8 - 8 = 0$
P2	1	4	18	$18 - 1 = 17$	$17 - 4 = 13$
P3	2	2	11	$11 - 2 = 9$	$9 - 2 = 7$
P4	3	1	9	$9 - 3 = 6$	$6 - 1 = 5$
P5	4	3	14	$14 - 4 = 10$	$10 - 3 = 7$

Average Waiting Time = $(0 + 13 + 7 + 5 + 7) / 5 = 6.4$ ms

Average Turnaround Time = $(8 + 17 + 9 + 6 + 10) / 5 = 10$ ms

Priority Scheduling

- Priority scheduling is a non-preemptive algorithm and one of the most common scheduling algorithms in batch systems.
- Each process is assigned a priority. Process with highest priority is to be executed first and so on.
- Processes with same priority are executed on first come first served basis.
- Priority can be decided based on memory requirements, time requirements or any other resource requirement.

Problem:

Consider the following set of processes with their arrival times and burst times:

Process	Arrival Time (ms)	Burst Time (ms)	Priority
P1	0	10	3
P2	1	1	1
P3	2	2	4
P4	3	1	5
P5	4	5	2

(Lower number = Higher priority)

Step-by-Step Execution

Step 1: Time = 0

Only **P1** has arrived → Start **P1**

Step 2: Time = 10 (P1 completes)

Processes arrived: P2, P3, P4, P5

Choose highest priority: **P2 (Priority 1)** → Execute

Step 3: Time = 11 (P2 completes)

Remaining: P3 (4), P4 (5), P5 (2)

Next highest: **P5 (Priority 2)** → Execute

Step 4: Time = 16 (P5 completes)

Remaining: P3 (Priority 4), P4 (Priority 5)

Next: **P3** → Execute

Step 5: Time = 18 (P3 completes)

Last: **P4** → Execute

Gantt Chart

P1	P2	P5	P3	P4	
0	10	11	16	18	19

Now calculate the values:

Process	Arrival Time(AT)	Burst Time(BT)	Priority	Completion Time(CT)	TAT = CT - AT	WT = TAT - BT
P1	0	10	3	10	$10 - 0 = 10$	$10 - 10 = 0$
P2	1	1	1	11	$11 - 1 = 10$	$10 - 1 = 9$
P3	2	2	4	18	$18 - 2 = 16$	$16 - 2 = 14$
P4	3	1	5	19	$19 - 3 = 16$	$16 - 1 = 15$
P5	4	5	2	16	$16 - 4 = 12$	$12 - 5 = 7$

Average Waiting Time = $(0 + 9 + 14 + 15 + 7) / 5 = 9 \text{ ms}$

Average Turnaround Time = $(10 + 10 + 16 + 16 + 12) / 5 = 12.8 \text{ ms}$

Shortest Remaining Time (SRT)

- Shortest remaining time (SRT) is the preemptive version of the SJN algorithm.
- The processor is allocated to the job closest to completion but it can be preempted by a newer ready job with shorter time to completion.
- Impossible to implement in interactive systems where required CPU time is not known.
- It is often used in batch environments where short jobs need to give preference.

Problem:

Consider the following set of processes with their arrival times and burst times:

Process	Arrival Time (ms)	Burst Time (ms)
P1	0	8
P2	1	4
P3	2	2
P4	3	1

Step-by-Step Execution

Time = 0

- Only P1 has arrived.
- So CPU starts running P1.
- Remaining time for P1 is now 7 ms.

Time = 1

- P2 arrives with burst time = 4.
- Compare P1 (7 ms) vs P2 (4 ms):
P2 has less time → So it takes over the CPU (preempts P1).
- Now P2 is running.
- Remaining time for P2 = 3 ms.

Time = 2

- P3 arrives with burst time = 2.
- Compare P2 (3 ms left) vs P3 (2 ms):
P3 has less time → So it preempts P2 and runs.
- Now P3 is running.
- Remaining time for P3 = 1 ms.

Time = 3

- P4 arrives with burst time = 1.
- Compare P3 (1 ms left) vs P4 (1 ms):
Both are equal. We continue with P3 (since it started first).
- P3 finishes now.

Time = 4

- Now check remaining processes:
 - P1 → 7 ms left
 - P2 → 3 ms left
 - P4 → 1 ms
- Smallest = P4 (1 ms) → So P4 runs and finishes.

Time = 5

- Remaining:
 - P1 → 7 ms
 - P2 → 3 ms
- Smallest = P2 (3 ms) → So P2 runs.

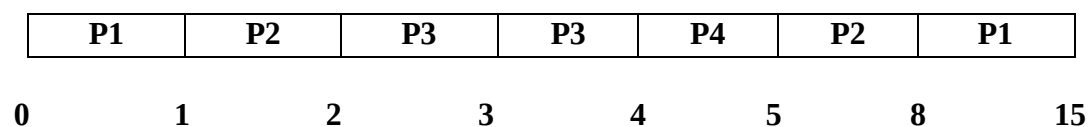
Time = 8

- P2 finishes.
- Only P1 is left.

Time = 8 to 15

- P1 runs and finishes at time = 15.

Gantt Chart



Now calculate the values:

Process	Arrival Time(AT)	Burst Time(BT)	Completion Time(CT)	TAT = CT – AT	WT = TAT – BT
P1	0	8	15	$15 - 0 = 15$	$15 - 8 = 7$
P2	1	4	8	$8 - 1 = 7$	$7 - 4 = 3$
P3	2	2	4	$4 - 2 = 2$	$2 - 2 = 0$
P4	3	1	5	$5 - 3 = 2$	$2 - 1 = 1$

Average Waiting Time: $(7 + 3 + 0 + 1) / 4 = 2.75$ ms

Average Turnaround Time: $(15 + 7 + 2 + 2) / 4 = 6.5$ ms

Round Robin(RR) Scheduling

- Each process is assigned a fixed time slot (time quantum).
- Processes are kept in a circular queue.
- If a process doesn't finish in its time quantum, it's moved to the end of the queue.

Problem:

Consider the following set of processes with their arrival times and burst times:

Process	Burst Time (ms)
P1	5
P2	15
P3	4
P4	7

Step-by-Step Execution

Step 1: Time = 0

- ◆ Queue = [P1, P2, P3, P4]
- ◆ P1 runs for 4 ms (needs 1 ms more)

Step 2: Time = 4

- ◆ Queue = [P2, P3, P4, P1]
- ◆ P2 runs for 4 ms (needs 11 ms more)

Step 3: Time = 8

- ◆ Queue = [P3, P4, P1, P2]
- ◆ P3 runs for 4 ms → ☒ Completed

Step 4: Time = 12

- ◆ Queue = [P4, P1, P2]
- ◆ P4 runs for 4 ms (needs 3 ms more)

Step 5: Time = 16

- ◆ Queue = [P1, P2, P4]
- ◆ P1 runs for 1 ms → ☒ Completed

Step 6: Time = 17

- ◆ Queue = [P2, P4]
- ◆ P2 runs for 4 ms (needs 7 ms more)

Step 7: Time = 21

◆ Queue = [P4, P2]

◆ P4 runs for 3 ms → ☒ Completed

Step 8: Time = 24

◆ Queue = [P2]

◆ P2 runs for 4 ms (needs 3 ms more)

Step 9: Time = 28

◆ Queue = [P2]

◆ P2 runs for 3 ms → ☒ Completed

Gantt Chart

P1	P2	P3	P4	P1	P2	P4	P2	P2	
0	4	8	12	16	17	21	24	28	31

Process	Arrival Time(AT)	Burst Time(BT)	Completion Time(CT)	TAT = CT - AT	WT = TAT - BT
P1	0	5	17	$17 - 0 = 17$	$17 - 5 = 12$
P2	0	15	31	$31 - 0 = 31$	$31 - 15 = 16$
P3	0	4	12	$12 - 0 = 12$	$12 - 4 = 8$
P4	0	7	24	$24 - 0 = 24$	$24 - 7 = 17$

Average Turnaround Time = $(17 + 31 + 12 + 24) / 4 = 21$ ms

Average Waiting Time = $(12 + 16 + 8 + 17) / 4 = 13.25$ ms

Multiple-Level Queue Scheduling

Multiple-Level Queue Scheduling is a CPU scheduling technique where processes are divided into different groups (queues) based on their type or priority. Each queue has its own scheduling algorithm, and processes do not move between queues. The queues are usually organized by importance (e.g., system processes > interactive > background jobs).

How It Works:

- The **CPU is shared among the queues**, typically using **fixed priority** (higher-priority queues are served first).
- **Each queue** can use its own method:
For example:
 - System queue → Round Robin
 - Interactive queue → FCFS
 - Batch queue → SJF
- Once a process is in a queue, it stays there.

Advantages:

1. Clear separation between process types.
2. Efficient handling of high-priority tasks.
3. Custom scheduling per queue.

Disadvantages:

1. Inflexible – no movement between queues.
2. Starvation for low-priority queues.
3. Complex to manage and tune.

IMPORTANT QUESTIONS:

SHORTS:

- 1) What is program and process?
- 2) What is process management?
- 3) What is process control block?
- 4) What is sequential process?
- 5) What is scheduler?
- 6) What is FCFS & SJN?
- 7) What are the 3 major activities of an operating system in regards to process management?
- 8) What is turnaround time and response time?
- 9) Difference between program and process?
- 10) What is preemptive and non-preemptive scheduling?
11. Define thread?

ESSAYS:

1. Explain the process state diagram?
2. Explain about Process Control Block?
3. Explain process creation & termination?
4. Explain the Threads & multi threading models?
5. Explain the relation between processes?
6. Describe scheduling queues and schedulers?
7. Explain cpu scheduling and scheduling criteria?
8. Explain various scheduling algorithms?
 - i. FCFS
 - ii. SJF
 - iii. ROUND ROBIN
 - iv. PRIORITY
 - v. MULTI LEVEL SCHEDULING
 - vi. SHORTEST REMAINING TIME